

BeaglePlay — Detailed Service-by-Service Operations Guide

Board: BeagleBone / BeaglePlay environment; Debian GNU/Linux 12 (Bookworm); ARM64; Linux 6.12.43-ti-arm64-r53 PREEMPT_RT.

Inventory captured: 8 August 2026. This guide is based on the supplied BeaglePlay inventory/application-details logs. It intentionally distinguishes **observed facts** from **engineering inference** and **unknowns not captured by the inventory**.

Ownership clarification: GNSS_Rover_Base.jar, gnss_gui_server, index.html, and the UI code are your own software. This guide therefore treats those as your application and concentrates on how Linux/systemd starts and supports them.

Important: The inventory contains 212 systemd unit files, but only 35 services were running at capture time and 3 were failed. Many of the 212 are static/template/alias units and are not independent running applications. This document gives detailed operational treatment to the active services, failed services, custom services/timers, and the important inactive custom units. It does not pretend that a package unit's internal ExecStart is known when the supplied inventory did not capture its full unit file.

How to use this document

Read Sections 1–4 first. Then use the individual service sections as a reference before changing anything. The most important columns are: program, user, device/port ownership, dependencies, failure behavior, and whether a new service can safely interact with it.

1. Executive Architecture

The operational model is:

Power ON → **bootloader/kernel** → **systemd** → **system targets** → **hardware/network services** → **custom services** → **your applications**.

Your Java application is launched through **gnss.service**:

systemd → gnss.service → /opt/gnss/start_gnss.sh → /usr/bin/java -jar /home/debian/GNSS_Rover_Base.jar

Your GUI server is launched independently:

systemd → gnss_gui_server.service → /home/debian/gnss_gui_server → TCP :8080

The inventory shows the GUI server listening on TCP 8080, the serial bridge on TCP 2000, and the MCU server on TCP 2001. The GNSS Java logs show it connects to the GNSS base at **192.168.10.100:2000** and to an NTRIP server. This is the key operational separation: your Java application consumes the serial bridge over TCP rather than being the owner of ttyS0 itself.

2. Service Classification

Class	Services	What it means
Your application	gnss.service; gnss_gui_server.service	These directly launch your own software.
Hardware/custom	gnss-watchdog; gnss_gpio; serial-bridge; mcu_server	Custom programs that interact with watchdog, GPIO, UART and TCP.

Class	Services	What it means
Network/custom	ip-manager; gnss-wifi-setup; gnss-wifi-connect; quectel-check; modem_init; modem_status; net-timesync	Product/network setup and monitoring.
Web/runtime	nginx; nodered; code-server; docker; containerd	Potentially important infrastructure; exact product usage must be verified before changing.
Core OS	systemd-journald; udevd; dbus; logind; networkd; timesyncd; getty; etc.	Linux operating environment; normally leave untouched.
Failed	bring-up-eth2; dnsmasq; networking	Currently failed according to the captured system state; must be investigated before relying on them.

3.1 gnss.service

Why it exists

Automatically starts and supervises your Java GNSS Base Station application. The service itself does not contain GNSS algorithms; it is the lifecycle wrapper around your script/JAR.

1. Who starts it

systemd pulls it in through multi-user.target. It is enabled. The unit explicitly wants network-online.target.

2. Exact command/program

/opt/gnss/start_gnss.sh. The captured cgroup shows that this script starts /usr/bin/java -jar /home/debian/GNSS_Rover_Base.jar.

3. Configuration files

Unit: /etc/systemd/system/gnss.service. Application/configuration paths observed include /opt/gnss and /home/debian/gnss/config/*.

4. User/root permissions

User=debian. UID/GID shown as 1000. This is important: the Java process is not running as root.

5. Hardware devices

Not directly claimed by the unit. The Java application connects to the GNSS base at 192.168.10.100:2000. ttyS0 is owned by serial-bridge, not by this service.

6. TCP/UDP ports

TCP :2000 is the observed GNSS serial-bridge endpoint used by your Java application. NTRIP network endpoint is used by your Java application but the remote port was not captured in a way that should be hard-coded here.

7. Files read/written

Observed configuration files include /home/debian/gnss/config/ip_port.txt, ip_setting.txt, save_gnss_receiver_settings.txt and version.txt; exact per-class file usage belongs to your application code.

8. Other services/dependencies

After=network-online.target; Wants=network-online.target. It also requires normal system targets/mounts. RequiresMountsFor=/opt/gnss is shown by systemd.

9. What happens if it stops

If manually stopped, your Java GNSS process is stopped and GNSS service functionality stops. The GUI server is a separate service and is not automatically stopped.

10. What happens if it crashes

Restart=always with RestartSec=10 means an unexpected exit is normally followed by a restart. SuccessExitStatus=143 treats 143 as a successful exit status.

11. Can a new service safely interact?

Yes, provided the new service does not compete for the same TCP port, files, receiver, or configuration. Prefer communication through a defined API/socket/file rather than editing this unit for unrelated functionality.

12. Should it be left untouched?

Leave it untouched for unrelated features. Modify it only when changing how your Java application is launched, its dependencies, resource limits, or restart policy.

Systemd unit — line-by-line interpretation

```
[Unit]
Description=GNSS Java Base Station Auto Start
After=network-online.target
Wants=network-online.target
```

```
[Service]
Type=simple
User=debian
WorkingDirectory=/opt/gnss
ExecStart=/opt/gnss/start_gnss.sh
Restart=always
RestartSec=10
SuccessExitStatus=143
```

```
[Install]
WantedBy=multi-user.target
```

Evidence level: High. The service definition, process tree, dependency graph and live journal were captured.

3.2 gnss_gui_server.service

Why it exists

Automatically runs your native GUI/web server independently from the Java GNSS application.

1. Who starts it

systemd via multi-user.target. Enabled.

2. Exact command/program

/home/debian/gnss_gui_server

3. Configuration files

Unit: /etc/systemd/system/gnss_gui_server.service. Your binary references ./gnss/.sys_cal.dat, ./gnss/config/*, ./gnss/ui/, and related GUI/log resources.

4. User/root permissions

User=debian. Not root.

5. Hardware devices

None explicitly stated in the unit. It is a network application.

6. TCP/UDP ports

Observed: TCP 0.0.0.0:8080 is owned by gnss_gui_server PID 721.

7. Files read/written

Observed GUI paths include ./gnss/ui/, ./gnss/config/update.flag, ipport.txt, password.txt, wifi_setting.txt, ip_port.txt, save_gnss_receiver_settings.txt and ./gnss/.sys_cal.dat. The exact read/write operation is application-code dependent.

8. Other services/dependencies

After=network.target. Normal system targets and journald are pulled in. No explicit Requires=gnss.service is present.

9. What happens if it stops

If stopped, the web GUI/API becomes unavailable on port 8080. The Java GNSS service continues unless your application architecture creates an indirect dependency.

10. What happens if it crashes

Restart=always; RestartSec=5. An unexpected exit is restarted after approximately 5 seconds.

11. Can a new service safely interact?

Yes. A new service may call its HTTP API if the API is designed for it, but it must not take TCP 8080 or overwrite the same config files.

12. Should it be left untouched?

Leave untouched unless changing the GUI server itself. Do not add unrelated functionality into this service just because it is already running.

Systemd unit — line-by-line interpretation

```
[Unit]
Description=GNSS GUI Server Service
After=network.target

[Service]
Type=simple
ExecStart=/home/debian/gnss_gui_server
WorkingDirectory=/home/debian
Restart=always
RestartSec=5
```

```
User=debian
```

```
[Install]  
WantedBy=multi-user.target
```

Evidence level: High for startup/port/permissions; exact endpoint behavior comes from your source code rather than systemd.

3.3 gnss-watchdog.service

Why it exists

Continuously services the hardware watchdog so the board can recover from a system/software failure.

1. Who starts it

systemd via multi-user.target. Enabled.

2. Exact command/program

ExecStartPre waits until /dev/watchdog1 exists; then /opt/gnss/watchdog runs.

3. Configuration files

Unit: /etc/systemd/system/gnss-watchdog.service.

4. User/root permissions

User=root because direct hardware watchdog access requires elevated permissions on this system.

5. Hardware devices

/dev/watchdog1.

6. TCP/UDP ports

No TCP/UDP port.

7. Files read/written

/dev/watchdog1 is the critical device. Journal contains periodic 'Watchdog kick' messages.

8. Other services/dependencies

After=multi-user.target; normal system targets are required.

9. What happens if it stops

If stopped and the watchdog remains armed, the board may reset when the watchdog timeout expires. Do not assume stopping it is harmless.

10. What happens if it crashes

Restart=always; RestartSec=2. If the process crashes, systemd attempts to restart it.

11. Can a new service safely interact?

Only interact by normal application-level mechanisms. Do not open/control /dev/watchdog1 from a new service unless you are intentionally redesigning the watchdog architecture.

12. Should it be left untouched?

Yes — leave untouched unless you are specifically changing board recovery/watchdog behavior.

Systemd unit — line-by-line interpretation

```
[Unit]  
Description=GNSS Hardware Watchdog Service  
After=multi-user.target  
  
[Service]  
Type=simple  
ExecStartPre=/bin/sh -c 'while [ ! -e /dev/watchdog1 ]; do sleep 1; done'  
ExecStart=/opt/gnss/watchdog  
Restart=always  
RestartSec=2  
User=root  
  
[Install]  
WantedBy=multi-user.target
```

Evidence level: High. The live process and repeated watchdog kicks were captured.

3.4 gnss_gpio.service

Why it exists

Runs the custom GNSS GPIO program. Inventory evidence shows libgpiod usage, gpiochip3 and a GNSS power-enable style signal name.

1. Who starts it

systemd via multi-user.target. Enabled.

2. Exact command/program

/opt/gnss/gnss_gpio

3. Configuration files

Unit: /etc/systemd/system/gnss_gpio.service. Exact GPIO configuration is embedded in/used by the program; the inventory did not provide a complete line-to-board schematic mapping.

4. User/root permissions

User=root.

5. Hardware devices

GPIO subsystem; binary strings reference gpiochip3 and GNSS GPIO names. Exact line number/physical pin must be verified before modification.

6. TCP/UDP ports

No network port.

7. Files read/written

Hardware GPIO state; journal mostly shows service lifecycle.

8. Other services/dependencies

After=network-online.target; Wants=network-online.target. Note: the program is GPIO-focused even though its unit waits for network-online.

9. What happens if it stops

If stopped, whatever GNSS GPIO state/control it maintains stops. Exact external effect depends on the GPIO logic and board wiring.

10. What happens if it crashes

Restart=always; RestartSec=5.

11. Can a new service safely interact?

Do not use the same GPIO line from another service until the line is mapped and ownership is confirmed.

12. Should it be left untouched?

Leave untouched. A new feature should use a separate GPIO line or a controlled IPC interface.

Systemd unit — line-by-line interpretation

```
[Unit]
Description=GNSS GPIO Service
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
ExecStart=/opt/gnss/gnss_gpio
WorkingDirectory=/opt/gnss
Restart=always
RestartSec=5
User=root

[Install]
WantedBy=multi-user.target
```

Evidence level: High for service behavior; medium for exact physical GPIO function because the inventory does not contain the full hardware schematic.

3.5 serial-bridge.service

Why it exists

Provides the Serial-to-LAN path between a network client and the UART device used for the GNSS/base receiver path.

1. Who starts it

systemd via multi-user.target. Enabled.

2. Exact command/program

/bin/bash -c "sleep 10; /usr/local/bin/serial_bridge"

3. Configuration files

Unit: /etc/systemd/system/serial-bridge.service. Binary references /dev/ttyS0 and SERIAL_PORT "/dev/ttyS0".

4. User/root permissions

User=root.

5. Hardware devices

/dev/ttyS0. Inventory shows it as a BeaglePlay mikroBUS UART device. It is a critical shared resource.

6. TCP/UDP ports

TCP 0.0.0.0:2000 is listening. The binary references TCP socket/listening behavior and PORT 2000.

7. Files read/written

Reads/writes the serial device; no product data file was established from the inventory. Logs go through the service/journal if emitted.

8. Other services/dependencies

After=network.target. The unit deliberately sleeps 10 seconds before launching the bridge.

9. What happens if it stops

If stopped, the TCP-to-serial path on port 2000 disappears; your Java application can no longer reach the base at 192.168.10.100:2000.

10. What happens if it crashes

Restart=always. Because the command contains sleep 10 followed by the bridge, a failed bridge is relaunched by systemd.

11. Can a new service safely interact?

Yes only as a TCP client using port 2000. Do NOT open ttyS0 directly or bind port 2000 from another application.

12. Should it be left untouched?

Leave untouched. This is a core interface for the GNSS application.

Systemd unit — line-by-line interpretation

```
[Unit]
Description=Serial-to-LAN Bridge
After=network.target

[Service]
ExecStart=/bin/bash -c "sleep 10; /usr/local/bin/serial_bridge"
Restart=always
User=root

[Install]
WantedBy=multi-user.target
```

Evidence level: High. Port 2000 and ttyS0 ownership were captured.

3.6 mcu_server.service

Why it exists

Provides a TCP server for an MCU attached to ttyS3.

1. Who starts it

systemd via multi-user.target. Enabled.

2. Exact command/program

/usr/local/bin/mcu_server

3. Configuration files

Unit: /etc/systemd/system/mcu_server.service. Binary references /dev/ttyS3.

4. User/root permissions

User=debian.

5. Hardware devices

/dev/ttyS3; inventory identifies it as play/mikrobus/i2c-uart2. The process table shows PID 727 mcu_server using ttyS3.

6. TCP/UDP ports

TCP 0.0.0.0:2001 is listening and owned by mcu_server PID 727.

7. Files read/written

Serial data through ttyS3; no product file ownership was conclusively established.

8. Other services/dependencies

After=network-online.target; Wants=network-online.target.

9. What happens if it stops

If stopped, the MCU TCP interface on port 2001 disappears.

10. What happens if it crashes

Restart=always; RestartSec=2.

11. Can a new service safely interact?

A new service may use TCP 2001 as a client if the protocol is intentionally shared, but must not open ttyS3 independently.

12. Should it be left untouched?

Leave untouched unless the MCU protocol is being changed.

Systemd unit — line-by-line interpretation

```
[Unit]
Description=MCU Server
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
ExecStart=/usr/local/bin/mcu_server
Restart=always
RestartSec=2
User=debian
WorkingDirectory=/usr/local/bin
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Evidence level: High.

3.7 ip-manager.service

Why it exists

Applies the configured GNSS IP address/port to eth0 and then restarts the GNSS service.

1. Who starts it

systemd via multi-user.target. Enabled.

2. Exact command/program

/opt/gnss/ip_manager

3. Configuration files

Unit: /etc/systemd/system/ip-manager.service. Application strings show /home/debian/gnss/config/ip_setting.txt and ip_port.txt.

4. User/root permissions

Unit does not specify User, so it runs with systemd's default service user context (normally root for a simple service without User=). The inventory shows it successfully running network commands.

5. Hardware devices

eth0.

6. TCP/UDP ports

No listening port of its own.

7. Files read/written

Reads ip_setting.txt and ip_port.txt; logs show IP cleanup/update and 'systemctl restart gnss.service'.

8. Other services/dependencies

After=network-online.target; Wants=network-online.target.

9. What happens if it stops

If stopped before completion, the configured additional IP may not be applied. Once completed, it is a oneshot service and remains active (exited).

10. What happens if it crashes

No Restart directive. If it fails, systemd does not automatically loop it.

11. Can a new service safely interact?

Do not have a new service modify eth0 addresses or restart gnss.service independently without coordination.

12. Should it be left untouched?

Leave untouched. This is a configuration utility and has a direct side effect on your GNSS application.

Systemd unit — line-by-line interpretation

```
[Unit]
Description=GNSS IP Manager Service
After=network-online.target
Wants=network-online.target

[Service]
Type=oneshot
ExecStart=/opt/gnss/ip_manager
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

Evidence level: High for unit behavior; exact IP algorithm comes from the captured program output.

3.8 gnss-wifi-setup.service

Why it exists

Prepares the Wi-Fi AP configuration, specifically setting the AP SSID from the device serial number.

1. Who starts it

systemd via multi-user.target; ordered after network-pre.target and before hostapd.service.

2. Exact command/program

/home/debian/wifi_setup/wifi_setup

3. Configuration files

Unit: /etc/systemd/system/gnss-wifi-setup.service. Program works in /home/debian and updates hostapd.conf.

4. User/root permissions

No User= is specified; therefore it runs with the default system service privileges (root unless changed elsewhere).

5. Hardware devices

wlan/AP configuration; no direct device node is specified.

6. TCP/UDP ports

No application TCP/UDP port of its own.

7. Files read/written

Reads device serial information; writes hostapd.conf. The journal explicitly says hostapd.conf was updated successfully.

8. Other services/dependencies

After=network-pre.target; Wants=network-pre.target; Before=hostapd.service.

9. What happens if it stops

If it stops after successful completion, no continuously running process is lost because it is a oneshot. If it fails before updating hostapd.conf, the AP may use an old/default SSID.

10. What happens if it crashes

No Restart; oneshot. Failure requires another invocation/reboot/manual start.

11. Can a new service safely interact?

Do not modify hostapd.conf from a competing feature while this setup mechanism is active.

12. Should it be left untouched?

Leave untouched unless changing the product's Wi-Fi AP identity behavior.

Systemd unit — line-by-line interpretation

```
[Unit]
Description=Set WiFi AP SSID from device serial number
After=network-pre.target
Before=hostapd.service
Wants=network-pre.target

[Service]
Type=oneshot
WorkingDirectory=/home/debian
ExecStart=/home/debian/wifi_setup/wifi_setup
RemainAfterExit=yes
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Evidence level: High.

3.9 gnss-wifi-connect.service

Why it exists

Attempts to connect wlan0 to an external Wi-Fi network for Internet access.

1. Who starts it

systemd via multi-user.target; wants and starts after NetworkManager.service.

2. Exact command/program

/home/debian/wifi_setup/wifi_connect

3. Configuration files

Unit: /etc/systemd/system/gnss-wifi-connect.service. Exact Wi-Fi profile/SSID logic is in the program.

4. User/root permissions

No User= is specified; default system service context applies.

5. Hardware devices

wlan0.

6. TCP/UDP ports

No dedicated listening port.

7. Files read/written

Wi-Fi connection profiles managed through the network stack. The current journal repeatedly reports attempts to connect to SSID/profile 'gnss'.

8. Other services/dependencies

After=NetworkManager.service; Wants=NetworkManager.service.

9. What happens if it stops

If stopped, automatic external-Wi-Fi reconnection stops. Other network interfaces may continue working.

10. What happens if it crashes

Restart=on-failure; RestartSec=5. The program itself also retries, according to the journal.

11. Can a new service safely interact?

A new service should not manipulate wlan0/network profiles independently without coordinating with this service and the active network manager.

12. Should it be left untouched?

Leave untouched until its current Wi-Fi error is understood. The inventory shows repeated 'invalid extra argument --' errors and failed connection attempts.

Systemd unit — line-by-line interpretation

```
[Unit]
Description=Connect BeaglePlay to external WiFi for internet access
After=NetworkManager.service
Wants=NetworkManager.service

[Service]
Type=simple
```

```
WorkingDirectory=/home/debian
ExecStart=/home/debian/wifi_setup/wifi_connect
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal
```

```
[Install]
WantedBy=multi-user.target
```

Evidence level: High for service behavior; medium for the exact Wi-Fi command sequence because it is inside the binary.

3.10 quectel-check.service

Why it exists

Continuously checks the Quectel EC200U cellular/network interface and attempts to keep/check eth2 connectivity.

1. Who starts it

systemd via multi-user.target.

2. Exact command/program

/bin/bash -c 'while true; do /usr/local/bin/quectel_check; sleep 10; done'

3. Configuration files

Unit: /etc/systemd/system/quectel-check.service. Binary references ttyUSB devices and eth2.

4. User/root permissions

No User= specified; default service privileges apply.

5. Hardware devices

Quectel EC200U-related ttyUSB* devices and eth2. Current eth2 is 192.168.225.33/24.

6. TCP/UDP ports

No dedicated listening port.

7. Files read/written

Program/log output and modem state. The inventory shows repeated 'eth2 already has IP' messages.

8. Other services/dependencies

After=multi-user.target.

9. What happens if it stops

If stopped, the 10-second recurring modem/network check loop stops.

10. What happens if it crashes

Restart=always. The shell loop itself is continuous; systemd restarts it if the shell exits.

11. Can a new service safely interact?

Do not independently reconfigure eth2 or the modem while this loop is managing it.

12. Should it be left untouched?

Leave untouched unless redesigning cellular connectivity.

Systemd unit — line-by-line interpretation

```
[Unit]
Description=Quectel EC200U Auto Connect Check
After=multi-user.target

[Service]
Type=simple
ExecStart=/bin/bash -c 'while true; do /usr/local/bin/quectel_check; sleep 10; done'
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Evidence level: High.

3.11 modem_init.service

Why it exists

Runs the modem ECM boot initialization script once after the multi-user target is reached.

1. Who starts it

systemd via multi-user.target.

2. Exact command/program

/usr/local/bin/modem_init.sh

3. Configuration files

Unit: /etc/systemd/system/modem_init.service; script is /usr/local/bin/modem_init.sh.

4. User/root permissions

No User= specified; default service privileges apply.

5. Hardware devices

Quectel modem/network interface, especially eth2. The inventory shows it performs ECM-related setup.

6. TCP/UDP ports

No dedicated port.

7. Files read/written

Script output goes to its command environment/logs; modem_full_log.txt is used by modem-related software.

8. Other services/dependencies

After=multi-user.target.

9. What happens if it stops

It is a oneshot initialization. Stopping after successful completion normally has no persistent process to kill.

10. What happens if it crashes

No Restart. Failure is not automatically retried.

11. Can a new service safely interact?

A new service should not redo modem ECM initialization unless the modem architecture is intentionally being changed.

12. Should it be left untouched?

Leave untouched.

Systemd unit — line-by-line interpretation

```
[Unit]
Description=Modem ECM Boot Initializer
After=multi-user.target

[Service]
Type=oneshot
ExecStart=/usr/local/bin/modem_init.sh

[Install]
WantedBy=multi-user.target
```

Evidence level: High for unit; exact script behavior is separate.

3.12 modem_status.service + modem_status.timer

Why it exists

Runs the modem_manager collector periodically and records modem status.

1. Who starts it

systemd timer triggers the service. Timer is enabled through multi-user.target.

2. Exact command/program

/usr/local/bin/modem_manager

3. Configuration files

Unit: /etc/systemd/system/modem_status.service; timer: /etc/systemd/system/modem_status.timer.

4. User/root permissions

Service has no User=, so default service privileges apply.

5. Hardware devices

Quectel modem ttyUSB* and eth2 are referenced by modem_manager.

6. TCP/UDP ports

No dedicated listening port.

7. Files read/written

Observed strings include /var/log/modem_full_log.txt and /home/debian/modem_status.txt.

8. Other services/dependencies

Timer: OnBootSec=30 and OnUnitActiveSec=60. The service itself is oneshot.

9. What happens if it stops

If the service is stopped, the next timer activation can run it again. If the timer is stopped, periodic status collection stops.

10. What happens if it crashes

No Restart on the oneshot service. Timer provides periodic execution.

11. Can a new service safely interact?

A new service can read status files if they are stable, but should not rewrite them. Avoid direct modem control unless coordinated.

12. Should it be left untouched?

Leave untouched.

Systemd unit — line-by-line interpretation

```
[Unit]
Description=Modem Status Collector

[Service]
Type=oneshot
ExecStart=/usr/local/bin/modem_manager

Timer:
OnBootSec=30
OnUnitActiveSec=60
```

Evidence level: High for scheduling and executable; exact modem_manager internals are program-level.

3.13 net-timesync.service

Why it exists

Synchronizes network time and then writes the system time to the hardware RTC.

1. Who starts it

systemd via multi-user.target after network-online.target.

2. Exact command/program

/usr/bin/bash -c "timedatectl set-ntp true && sleep 10 && hwclock -w"

3. Configuration files

No separate product config file captured.

4. User/root permissions

Default system service privileges.

5. Hardware devices

System clock and hardware RTC.

6. TCP/UDP ports

No network listening port.

7. Files read/written

RTC hardware state is modified through hwclock.

8. Other services/dependencies

After=network-online.target; Wants=network-online.target.

9. What happens if it stops

If stopped, the scheduled/manual synchronization action does not complete.

10. What happens if it crashes

No Restart. It is a oneshot.

11. Can a new service safely interact?

A new service should generally consume system time rather than trying to set the RTC itself.

12. Should it be left untouched?

Leave untouched.

Systemd unit — line-by-line interpretation

```
[Unit]
Description=Sync time when network comes online
After=network-online.target
Wants=network-online.target

[Service]
Type=oneshot
ExecStart=/usr/bin/bash -c "timedatectl set-ntp true && sleep 10 && hwclock -w"

[Install]
WantedBy=multi-user.target
```

Evidence level: High.

3.14 daily-timesync.service + timer

Why it exists

Performs daily NTP enablement and RTC synchronization.

1. Who starts it

systemd timer.

2. Exact command/program

/usr/bin/bash -c "timedatectl set-ntp true && sleep 15 && hwclock -w"

3. Configuration files

Unit /etc/systemd/system/daily-timesync.service; timer /etc/systemd/system/daily-timesync.timer.

4. User/root permissions

Default system privileges.

5. Hardware devices

System clock/RTC.

6. TCP/UDP ports

No listening port.

7. Files read/written

Hardware RTC is written.

8. Other services/dependencies

Timer OnCalendar is 08:00 according to the unit text.

9. What happens if it stops

If the service doesn't run, daily RTC update is skipped.

10. What happens if it crashes

No Restart on oneshot; timer is the scheduler.

11. Can a new service safely interact?

New service should not modify system time/RTC unless it has a strong requirement.

12. Should it be left untouched?

Leave untouched.

Systemd unit — line-by-line interpretation

```
daily-timesync.service:
Type=oneshot
ExecStart=/usr/bin/bash -c "timedatectl set-ntp true && sleep 15 && hwclock -w"

daily-timesync.timer:
OnCalendar=*-*-* 08:00:00
Persistent=true
```

Evidence level: High.

3.15 daily-reboot.service + timer

Why it exists

Automatically reboots the entire BeaglePlay according to its timer.

1. Who starts it

systemd timer.

2. Exact command/program

/usr/bin/systemctl reboot

3. Configuration files

Unit /etc/systemd/system/daily-reboot.service; timer /etc/systemd/system/daily-reboot.timer.

4. User/root permissions

Default system privileges; reboot requires system authority.

5. Hardware devices

Whole system.

6. TCP/UDP ports

No network port.

7. Files read/written

No data files; it invokes systemctl reboot.

8. Other services/dependencies

Timer OnCalendar is captured as 16:06:00 in the inventory, despite the unit Description saying '1 AM'. The actual OnCalendar value is the one systemd uses.

9. What happens if it stops

When timer fires, the whole board reboots. All services stop and then start again during the next boot.

10. What happens if it crashes

No Restart; the timer controls recurrence.

11. Can a new service safely interact?

A new service cannot safely assume uninterrupted uptime. Long-running jobs must tolerate reboot.

12. Should it be left untouched?

Do not modify or remove this until the intended product reboot schedule is confirmed. The Description and actual schedule disagree.

Systemd unit — line-by-line interpretation

```
daily-reboot.service:
Type=oneshot
ExecStart=/usr/bin/systemctl reboot
```

```
daily-reboot.timer:
Description=Reboot system daily at 1 AM
OnCalendar=*-*-* 16:06:00
Persistent=true
```

Evidence level: High. The schedule discrepancy is directly visible in the supplied configuration.

4. Active Standard/Infrastructure Services — operational reference

For these services the inventory provides names/descriptions, active state, and in many cases ports/logs, but not necessarily the complete unit-file contents. Therefore the entries below do not invent an ExecStart or configuration path when the supplied log did not capture it.

4.1 avahi-daemon.service

Why it exists: mDNS/DNS-SD service discovery on the local network.

Who/when starts it: Active/running; enabled.

Exact command/program: Not captured in the application-details custom-unit section; package-provided unit.

Configuration: Typically system networking; exact process arguments not captured.

Permissions: System service; package-defined user/privileges not captured here.

Hardware/resources: Network multicast.

TCP/UDP ports: UDP 5353 on IPv4 and IPv6 observed.

Files: Avahi system configuration; exact file ownership not captured.

Dependencies: Normal OS/network dependencies.

If stopped: Local service discovery stops; normal IP networking may continue.

If it crashes: Package/systemd behavior; exact restart policy not captured.

Safe interaction: Use normal hostname/service discovery APIs; do not bind UDP 5353.

Recommendation: Generally leave untouched.

4.2 bb-usb-gadgets.service

Why it exists: BeagleBoard.org USB gadget setup.

Who/when starts it: Active/running; enabled.

Exact command/program: Package/vendor unit; exact ExecStart not captured.

Configuration: USB gadget configuration.

Permissions: System privileges.

Hardware/resources: USB gadget controller.

TCP/UDP ports: No product TCP port established.

Files: Vendor USB gadget config.

Dependencies: Kernel/udev/USB stack.

If stopped: USB gadget functions may disappear.

If it crashes: Depends on package unit behavior.

Safe interaction: Do not manipulate the same USB gadget configuration casually.

Recommendation: Leave untouched.

4.3 code-server@debian.service

Why it exists: Runs code-server for remote development/editing.

Who/when starts it: Active/running.

Exact command/program: Template unit /lib/systemd/system/code-server@.service; exact ExecStart not captured.

Configuration: code-server user configuration; exact path not captured.

Permissions: Instance is @debian.

Hardware/resources: Network.

TCP/UDP ports: No port ownership was safely assigned from the captured listener list; do not guess.

Files: code-server config.

Dependencies: User/session and network.

If stopped: Remote code-server access stops.

If it crashes: Template controls restart behavior; not captured.

Safe interaction: Use its configured endpoint; don't reuse its port.

Recommendation: Leave untouched unless intentionally removing remote IDE.

4.4 containerd.service

Why it exists: Container runtime used by Docker.

Who/when starts it: Active/running; enabled.

Exact command/program: Package service; exact command not captured.

Configuration: /etc/containerd likely, but not established in the supplied source; do not treat as confirmed.

Permissions: System/package user context not captured.

Hardware/resources: Container namespaces/cgroups.

TCP/UDP ports: No direct listener assigned.

Files: Container runtime configuration.

Dependencies: Docker depends on container runtime.

If stopped: Containers may fail to start/manage.

If it crashes: Package unit behavior.

Safe interaction: New service can coexist; do not alter container runtime configuration without checking containers.

Recommendation: Leave untouched.

4.5 cron.service

Why it exists: Traditional scheduled-task daemon.

Who/when starts it: Active/running.

Exact command/program: Package service.

Configuration: Cron schedules in system cron locations; exact active jobs not fully enumerated here.

Permissions: System privileges as required.

Hardware/resources: No dedicated hardware.

TCP/UDP ports: No dedicated product port.

Files: /etc/crontab and cron directories are typical; exact active tasks require separate inspection.

Dependencies: OS scheduling.

If stopped: Cron jobs stop running.

If it crashes: Package behavior.

Safe interaction: A new feature can use cron, but systemd timers are preferred for service lifecycle.

Recommendation: Usually leave untouched.

4.6 dbus.service

Why it exists: System D-Bus IPC bus.

Who/when starts it: Active/running.

Exact command/program: System service; exact command not captured here.

Configuration: D-Bus configuration.

Permissions: System service.

Hardware/resources: IPC bus.

TCP/UDP ports: No product port.

Files: /etc/dbus-1 and runtime bus; exact file usage not captured.

Dependencies: Core OS IPC.

If stopped: Many desktop/system control functions can fail.

If it crashes: System-managed.

Safe interaction: Applications may use D-Bus APIs; do not replace or modify the bus.

Recommendation: Leave untouched.

4.7 docker.service

Why it exists: Docker Application Container Engine.

Who/when starts it: Active/running; enabled.

Exact command/program: Package unit; exact command not captured.

Configuration: Docker configuration; inventory includes /etc/default/docker.

Permissions: System/package privileges.

Hardware/resources: Container/network namespaces.

TCP/UDP ports: Docker normally uses a Unix socket; no product TCP listener assigned in the captured list.

Files: /etc/default/docker and Docker runtime state.

Dependencies: containerd and kernel namespaces.

If stopped: Container management stops; running containers may be affected.

If it crashes: Package unit behavior.

Safe interaction: New service can coexist; inspect docker containers before changing ports/interfaces.

Recommendation: Leave untouched.

4.8 getty@tty1.service

Why it exists: Provides the local Linux text console login on tty1.

Who/when starts it: Active/running.

Exact command/program: Template getty@.service.

Configuration: Getty/system console configuration.

Permissions: root-spawned login process with user login afterward; exact unit not captured.

Hardware/resources: tty1.

TCP/UDP ports: No network port.

Files: /etc/getty configuration as package-defined.

Dependencies: Core system/console.

If stopped: Local console login unavailable.

If it crashes: Template behavior.

Safe interaction: Do not use tty1 for application serial communication.

Recommendation: Leave untouched.

4.9 hostapd.service

Why it exists: Provides the Wi-Fi/Ethernet access point/authentication service.

Who/when starts it: Active/running; enabled.

Exact command/program: Package unit; gnss-wifi-setup is explicitly Before=hostapd.service.

Configuration: hostapd.conf is explicitly modified by gnss-wifi-setup.

Permissions: System privileges.

Hardware/resources: Wi-Fi AP interface (ap0 is present in interface list).

TCP/UDP ports: DHCP/AP-related UDP 67 is observed on ap0; hostapd itself is not the owner of DHCP.

Files: /etc/hostapd/hostapd.conf is updated by your Wi-Fi setup program.

Dependencies: network-pre / wireless stack.

If stopped: Device AP may disappear or authentication stops.

If it crashes: Package behavior.

Safe interaction: Do not compete for ap0 or rewrite hostapd.conf without coordination.

Recommendation: Leave untouched.

4.10 iwd.service

Why it exists: Wireless service/daemon.

Who/when starts it: Active/running; enabled.

Exact command/program: Package service; exact command not captured.

Configuration: iwd configuration/state.

Permissions: System service.

Hardware/resources: Wi-Fi wlan0.

TCP/UDP ports: No dedicated product port established.

Files: /etc/iwd likely, but not confirmed from source.

Dependencies: Network/wireless stack.

If stopped: Wireless management may stop.

If it crashes: Package behavior.

Safe interaction: Do not independently manage wlan0 while iwd/network stack is active.

Recommendation: Leave untouched.

4.11 mender-client.service

Why it exists: Mender OTA update client.

Who/when starts it: Active/running; enabled.

Exact command/program: Package unit.

Configuration: Mender configuration; exact file not captured.

Permissions: Package-defined privileges.

Hardware/resources: Network/storage.

TCP/UDP ports: No local listener assigned.

Files: Mender client state/config.

Dependencies: Network and storage.

If stopped: OTA updates and related management stop.

If it crashes: Package behavior.

Safe interaction: A new service must tolerate OTA-triggered restart/reboot/update operations.

Recommendation: Leave untouched.

4.12 NetworkManager.service

Why it exists: Network management daemon. Note: unit-file list says disabled, but the live running list says active; custom gnss-wifi-connect explicitly Wants/After it.

Who/when starts it: Active/running in captured state.

Exact command/program: Package service.

Configuration: NetworkManager configuration.

Permissions: System service.

Hardware/resources: eth/wlan/modem network devices.

TCP/UDP ports: No dedicated product port.

Files: /etc/NetworkManager and connection profiles are likely; exact active profile paths not captured.

Dependencies: Network interfaces.

If stopped: Network configuration changes may stop or change.

If it crashes: Package behavior.

Safe interaction: Do not modify interfaces/profiles from a new service without coordinating with the active network manager.

Recommendation: Leave untouched until network architecture is fully mapped.

4.13 nginx.service

Why it exists: Web server/reverse proxy.

Who/when starts it: Active/running; enabled.

Exact command/program: Package service; exact ExecStart not captured.

Configuration: nginx configuration; exact active server blocks not captured.

Permissions: Package-defined privileges.

Hardware/resources: Network.

TCP/UDP ports: TCP 80 observed on 0.0.0.0.

Files: /etc/nginx configuration is implied but not directly mapped to your GUI by the inventory.

Dependencies: Network.

If stopped: HTTP access through nginx stops.

If it crashes: Package behavior.

Safe interaction: New HTTP service must not use TCP 80; determine whether nginx proxies to your GUI server.

Recommendation: Leave untouched until proxy mapping is documented.

4.14 nodered.service

Why it exists: Node-RED graphical automation runtime.

Who/when starts it: Active/running; enabled.

Exact command/program: Package/custom unit; exact ExecStart not captured.

Configuration: Node-RED settings/flows.

Permissions: Package-defined user; logs show Node-RED PID 1189.

Hardware/resources: Network.

TCP/UDP ports: TCP 1880 observed; log says server running at 127.0.0.1:1880.

Files: Node-RED flow/settings files; exact paths not captured.

Dependencies: Network and runtime.

If stopped: Node-RED flows stop.

If it crashes: Package/custom unit behavior.

Safe interaction: A new service can call Node-RED APIs if intentionally designed; don't reuse 1880.

Recommendation: Leave untouched.

4.15 polkit.service

Why it exists: Authorization policy daemon used by system components.

Who/when starts it: Active/running.

Exact command/program: System package.

Configuration: polkit rules/config.

Permissions: System service.

Hardware/resources: D-Bus.

TCP/UDP ports: No product port.

Files: /etc/polkit-1 rules/config.

Dependencies: D-Bus/system authorization.

If stopped: Some privileged desktop/system actions may fail.

If it crashes: System behavior.

Safe interaction: Do not add broad privilege rules just to make a new service work; prefer proper systemd User/Group/device permissions.

Recommendation: Leave untouched.

4.16 rtkit-daemon.service

Why it exists: RealtimeKit scheduling policy service.

Who/when starts it: Active/running.

Exact command/program: Package service.

Configuration: System/package configuration.

Permissions: System user.

Hardware/resources: Kernel scheduling.

TCP/UDP ports: No product port.

Files: System runtime/config.

Dependencies: DBus/system scheduling.

If stopped: Realtime client policy operations may fail.

If it crashes: Package behavior.

Safe interaction: Normally no interaction required.

Recommendation: Leave untouched.

4.17 serial-getty@ttyGS0.service

Why it exists: Provides a login/getty on the USB gadget serial console ttyGS0.

Who/when starts it: Active/running.

Exact command/program: Template serial-getty@.service.

Configuration: Getty configuration.

Permissions: System login service.

Hardware/resources: /dev/ttyGS0.

TCP/UDP ports: No network port.

Files: Getty/system config.

Dependencies: getty target.

If stopped: USB serial console login unavailable.

If it crashes: Template behavior.

Safe interaction: Do not use ttyGS0 for application traffic unless you intentionally disable the console.

Recommendation: Leave untouched.

4.18 serial-getty@ttyS2.service

Why it exists: Provides a login/getty on ttyS2.

Who/when starts it: Active/running.

Exact command/program: Template serial-getty@.service.

Configuration: Getty configuration.

Permissions: System login service.

Hardware/resources: /dev/ttyS2.

TCP/UDP ports: No network port.

Files: Getty/system config.

Dependencies: getty target.

If stopped: ttyS2 login unavailable.

If it crashes: Template behavior.

Safe interaction: Do not reuse ttyS2 without removing/moving the getty configuration deliberately.

Recommendation: Leave untouched.

4.19 ssh.service

Why it exists: OpenSSH server for remote administration.

Who/when starts it: Active/running; enabled.

Exact command/program: /usr/sbin/sshd -D \$SSHD_OPTS; sshd -t is run before start according to the captured alias unit.

Configuration: /etc/ssh/sshd_config and /etc/default/ssh.

Permissions: root/master process with privilege separation; exact runtime users not captured.

Hardware/resources: Network interface.

TCP/UDP ports: TCP 22 observed on IPv4 and IPv6.

Files: /etc/ssh and host keys.

Dependencies: network.target; auditd condition in captured unit.

If stopped: Remote SSH administration is lost.

If it crashes: Restart=on-failure in captured unit.

Safe interaction: Do not use TCP 22 for a new application.

Recommendation: Leave untouched.

4.20 systemd-journald.service

Why it exists: Collects service/system logs.

Who/when starts it: Active/running.

Exact command/program: systemd internal service.

Configuration: journald.conf if customized; exact config not captured.

Permissions: systemd service.

Hardware/resources: Kernel/log sockets.

TCP/UDP ports: No product port.

Files: Journal storage.

Dependencies: Core system.

If stopped: Logging/diagnostics become unavailable or degraded.

If it crashes: System-managed.

Safe interaction: All new services should use journald unless a separate logging requirement exists.

Recommendation: Leave untouched.

4.21 systemd-logind.service

Why it exists: Manages user sessions, seats and login state.

Who/when starts it: Active/running.

Exact command/program: systemd internal.

Configuration: logind.conf if customized.

Permissions: systemd service.

Hardware/resources: D-Bus/udev.

TCP/UDP ports: No product port.

Files: /etc/systemd/logind.conf if customized.

Dependencies: D-Bus/udev.

If stopped: Session management may fail.

If it crashes: System-managed.

Safe interaction: No application interaction normally needed.

Recommendation: Leave untouched.

4.22 systemd-networkd.service

Why it exists: Low-level Linux network configuration daemon.

Who/when starts it: Active/running; enabled. The inventory also shows NetworkManager active, so the network architecture is mixed and must be handled carefully.

Exact command/program: /lib/systemd/system/systemd-networkd.service; exact ExecStart is captured in the alias unit as /lib/systemd/systemd-networkd.

Configuration: /etc/systemd/network and generated configuration; exact active files not fully captured.

Permissions: User=systemd-network in captured unit.

Hardware/resources: eth0/eth1/eth2/wlan/ap interfaces depending on configuration.

TCP/UDP ports: No dedicated product port.

Files: Networkd configuration/runtime.

Dependencies: network-pre/network targets.

If stopped: Network configuration may be disrupted.

If it crashes: Restart=on-failure in captured unit.

Safe interaction: Do not independently manipulate interfaces without understanding networkd + NetworkManager coexistence.

Recommendation: Leave untouched.

4.23 systemd-timesyncd.service

Why it exists: Network time synchronization daemon.

Who/when starts it: Active/running; enabled.

Exact command/program: /lib/systemd/systemd-timesyncd in captured alias unit.

Configuration: timesyncd configuration/state.

Permissions: User=systemd-timesync in captured unit.

Hardware/resources: System clock.

TCP/UDP ports: No product port.

Files: timesyncd config/state.

Dependencies: sysinit/time targets.

If stopped: Network time synchronization stops.

If it crashes: Restart=always in captured unit.

Safe interaction: New services should consume time, not compete to set it.

Recommendation: Leave untouched.

4.24 systemd-udevd.service

Why it exists: Creates/manages device nodes and reacts to hardware events.

Who/when starts it: Active/running.

Exact command/program: systemd internal daemon.

Configuration: udev rules under /etc/udev and /lib/udev; exact rules not enumerated.

Permissions: systemd/udev service.

Hardware/resources: All hotplug devices.

TCP/UDP ports: No product port.

Files: udev rules and device database.

Dependencies: Kernel/device stack.

If stopped: New devices may not be created/configured correctly.

If it crashes: System-managed.

Safe interaction: New hardware integration should add proper udev rules rather than bypassing udev.

Recommendation: Leave untouched.

4.25 udisks2.service

Why it exists: Disk/storage device management.

Who/when starts it: Active/running; enabled.

Exact command/program: Package service.

Configuration: udisks configuration.

Permissions: System service.

Hardware/resources: Storage devices.

TCP/UDP ports: No product port.

Files: udisks config/runtime.

Dependencies: DBus/storage.

If stopped: Desktop/storage management may fail.

If it crashes: Package behavior.

Safe interaction: Not relevant to ordinary GNSS services; avoid modifying.

Recommendation: Leave untouched.

4.26 unattended-upgrades.service

Why it exists: Debian unattended package-update/shutdown support.

Who/when starts it: Active/running at capture.

Exact command/program: Package service; description in running list is 'Unattended Upgrades Shutdown'.

Configuration: APT unattended-upgrades configuration.

Permissions: System privileges.

Hardware/resources: Storage/network/package manager.

TCP/UDP ports: No product port.

Files: APT configuration/logs.

Dependencies: APT/package lifecycle.

If stopped: May affect package updates and potentially reboot/update behavior.

If it crashes: Package behavior.

Safe interaction: New service should survive package updates and reboots.

Recommendation: Leave untouched unless maintenance policy is intentionally changed.

4.27 user@1000.service

Why it exists: systemd user manager for the debian user.

Who/when starts it: Active/running.

Exact command/program: systemd template user@.service.

Configuration: User-session configuration.

Permissions: UID 1000/debian.

Hardware/resources: User session/runtime.

TCP/UDP ports: No product port.

Files: User runtime directory/config.

Dependencies: PAM/login/user manager.

If stopped: User-session services/processes may be affected.

If it crashes: System-managed.

Safe interaction: Do not put critical system hardware services here; use system services instead.

Recommendation: Leave untouched.

5. Failed Services — do not ignore these

5. Failed Services — do not ignore these

5.1 bring-up-eth2.service

What the inventory establishes: Unit: /etc/systemd/system/bring-up-eth2.service.

[Unit] Description=Bring up eth2 if available; After=network.target.

[Service] ExecStart=/sbin/ifup eth2; Restart=on-failure.

[Install] WantedBy=multi-user.target.

Impact: Purpose: explicitly bring up eth2. It is currently failed. The board nevertheless has eth2 UP with 192.168.225.33/24 at capture time, so another mechanism may have brought eth2 up.

Engineering recommendation: Impact: cellular/network behavior may still work, but this failed unit indicates an inconsistent network startup design. Do not simply disable/delete it until the EC200U path is mapped.

Interaction rule: New services should not manipulate eth2 until this conflict is understood.

5.2 dnsmasq.service

What the inventory establishes: Failed. Its package description is a lightweight DHCP/caching DNS server. The inventory shows hostapd/ap0 and UDP 67 on ap0, so dnsmasq may be part of the AP/DHCP design, but the exact active configuration/owner should be verified.

Impact: Impact: DHCP/DNS functionality supplied by dnsmasq may be unavailable. Because ap0 DHCP traffic is observed, investigate before assuming dnsmasq is irrelevant.

Engineering recommendation: Do not replace or restart it blindly; map hostapd, dnsmasq, ap0 and DHCP first.

5.3 networking.service

What the inventory establishes: Failed. This is the traditional ifupdown network service. The system is also running systemd-networkd and reports NetworkManager active. Therefore the failure may be a legacy/unused network-management path rather than a complete network outage.

Impact: Impact: depends on which interfaces are configured through ifupdown. The board currently has eth0/eth2 configured, so other network managers are clearly involved.

Engineering recommendation: Do not 'fix' it blindly. First determine which interfaces and services actually depend on networking.service.

6. Network and Port Ownership Map

Port/interface	Observed owner/function	Why it matters for a new service
TCP 8080	gnss_gui_server PID 721	Your GUI server. Do not reuse.
TCP 2000	Serial-to-LAN bridge; GNSS Java connects to 192.168.10.100:2000	Critical GNSS receiver/base path. Do not bind another server to 2000.
TCP 2001	mcu_server PID 727	MCU interface. Do not reuse.
TCP 1880	Node-RED	Do not reuse unless Node-RED is intentionally changed.
TCP 3000	node process	Ownership must be identified before reuse.
TCP 22	SSH	Never reuse.
TCP 80	nginx	Do not reuse; determine nginx routing.
UDP 5353	mDNS/Avahi	Do not bind unless implementing mDNS intentionally.
UDP 67 on ap0	DHCP/AP path	Do not interfere with AP/DHCP.
eth0	10.10.0.1/24 and 192.168.10.100/24	Critical GNSS/network interface.
eth2	192.168.225.33/24	Quectel/EC200U network path.
wlan0	DOWN at capture	Managed by Wi-Fi stack; do not assume it is free.
ap0	DOWN at capture	AP interface; hostapd/dnsmasq involved.

The listener list is a snapshot. Before deployment of a new service, run ss -lntup again on the live board. A process may open a port dynamically after startup.

7. Serial Resource Ownership

Device	Observed mapping	Observed owner	Rule for new service
/dev/ttyS0	BeaglePlay mikroBUS UART	serial_bridge	Do not open directly; use TCP :2000 if you need the GNSS serial path.
/dev/ttyS3	BeaglePlay mikroBUS i2c-uart2	mcu_server PID 727	Do not open directly; use TCP :2001 if the protocol permits.
/dev/ttyS2	Serial console	serial-getty@ttyS2	Do not reuse without deliberately changing getty configuration.
/dev/ttyGS0	USB gadget serial console	serial-getty@ttyGS0	Do not reuse casually.
/dev/ttyUSB0..3	Quectel/modem-related USB serial devices	modem services/programs	Do not assume a free port; map each interface before use.

8. What 'enabled', 'active', 'running', 'exited' and 'failed' mean

enabled: systemd has a boot-time relationship that can start the unit. It does not mean the process is currently running.

active (running): a process associated with the service is currently running.

active (exited): a oneshot program finished successfully and systemd considers the unit active because RemainAfterExit=yes is used. ip-manager and gnss-wifi-setup are examples.

failed: the unit's last activation failed or was unsuccessful. It does not necessarily mean the entire board function is broken.

static: normally cannot be enabled directly; another unit pulls it in or it is a dependency.

alias: another name pointing to a service. Do not treat aliases as separate applications.

9. How Your New Service Should Be Designed

Recommended isolation model:

/opt/my_feature/my_feature

/opt/my_feature/config/

/opt/my_feature/logs/

/etc/systemd/system/my-feature.service

Good: new_service → TCP client → serial-bridge :2000.

Dangerous: new_service → open /dev/ttyS0 while serial_bridge already owns it.

Good: new_service → HTTP client → your GUI server :8080 if you intentionally expose an API.

Dangerous: new_service → bind :8080.

Good: new_service → read a status file designed for sharing.

Dangerous: new_service → continuously rewrite a file owned by ip-manager or the GUI server.

Good: new service has After=network-online.target if it truly requires network availability.

Dangerous: adding Requires=gnss.service when the new feature is unrelated, thereby making the new service tightly coupled to the GNSS application.

10. Safe Deployment Procedure for a New Service

1. Back up the current service/configuration files.
2. Confirm no existing service owns the intended port/device/file.
3. Create the executable under a dedicated directory.
4. Run the executable manually as the intended user before creating the systemd unit.
5. Create the .service file under /etc/systemd/system/.
6. Run `systemctl daemon-reload`.
7. Run `systemctl start my-feature.service` and inspect logs.
8. Run `systemctl status my-feature.service`.
9. Verify `ss -lntup`, serial devices, GPIO state and application behavior.
10. Only after testing, run `systemctl enable my-feature.service`.

11. Re-check `systemctl --failed` and your existing GNSS/GUI services.

12. Reboot only during a controlled maintenance/test window because this board has a daily reboot timer.

11. Service Inspection Checklist — use before touching ANY existing service

```
systemctl status SERVICE
systemctl cat SERVICE
systemctl show SERVICE
systemctl list-dependencies SERVICE
systemctl list-dependencies --reverse SERVICE
journalctl -u SERVICE -b --no-pager
systemctl is-enabled SERVICE
systemctl is-active SERVICE
ss -lntup
ls -l /dev/ttyS* /dev/ttyUSB* /dev/ttyGS*
sudo lsof /dev/ttyS0
sudo lsof /dev/ttyS3
systemctl --failed
```

For your new service, record the output of all of these before and after deployment. This gives you a baseline and makes rollback much safer.

12. Important Findings from the Current Inventory

- Your Java application is correctly separated behind `gnss.service` and runs as `debian`, not `root`.
- Your GUI server is separately managed by `gnss_gui_server.service` and listens on TCP 8080.
- `serial-bridge` owns `ttyS0` and exposes TCP 2000; your Java logs show connection to `192.168.10.100:2000`.
- `mcu_server` owns `ttyS3` and exposes TCP 2001.
- `gnss-watchdog` directly owns the hardware watchdog device `/dev/watchdog1`.
- `gnss_gpio` runs as `root` and is a hardware GPIO consumer.
- `ip-manager` can restart `gnss.service` after applying IP configuration. Therefore it is not just a passive configuration reader.
- `gnss-wifi-setup` modifies `hostapd.conf` and is explicitly ordered before `hostapd`.
- `gnss-wifi-connect` currently reports repeated Wi-Fi connection failures and 'invalid extra argument --' errors. This is an existing issue worth investigating before changing Wi-Fi behavior.
- The system has both `systemd-networkd` and an active `NetworkManager` service in the captured state, while `networking.service` is failed. This mixed network-management architecture must be understood before adding network configuration logic.
- `daily-reboot.timer` contains `OnCalendar=*-*-* 16:06:00` while its description says '1 AM'. This discrepancy must be resolved before assuming the intended reboot schedule.
- At capture time, `eth0` had `10.10.0.1/24` and `192.168.10.100/24`, while `eth2` had `192.168.225.33/24`. `wlan0` and `ap0` were down at the instant of the interface snapshot.

13. Final Engineering Rule

Do not think of the BeaglePlay as one application. Think of it as a collection of independently managed processes sharing a small number of critical resources. The safest extension strategy is to preserve those ownership boundaries. In particular: one owner per UART, one owner per listening TCP/UDP port, one clear owner for each GPIO line, one clear writer for each configuration file, and explicit `systemd` dependencies only where a real dependency exists.

14. Evidence / Unknowns

Where this document says 'not captured' or 'must verify', that is intentional. The supplied inventory contains a very large amount of output, but not every package unit's complete source configuration. I have not invented paths, ports, users, or dependencies simply because they are common on Debian. For production changes, those items should be verified on the live board with `systemctl cat/show`, `ss`, `lsof/fuser`, `udevadm`, and targeted `journalctl` commands.